

COMPUTER ARITHMETIC

Module.4

CDA 3103C

Mesut Ozdag, Ph.D.

Multiplying by a Constant

- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
- Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.

Multiplying by a Constant

- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
 - Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.
-
- $4 * 2 = 8$
 - $0100 * 0010 = 1000$
 - $0100 \ll 1 = 1000$

Multiplying by a Constant

- Some compilers replace multiplies by short constants with a series of shifts and adds. Because one bit to the left represents a number twice as large in base 2, shifting the bits left has the same effect as multiplying by a power of 2.
 - Almost every compiler will perform the strength reduction optimization of substituting a left shift for a multiply by a power of 2.
-
- $2 * 4 = 8$
 - $0010 * 0100 = 1000$
 - $0010 \ll 2 = 1000$

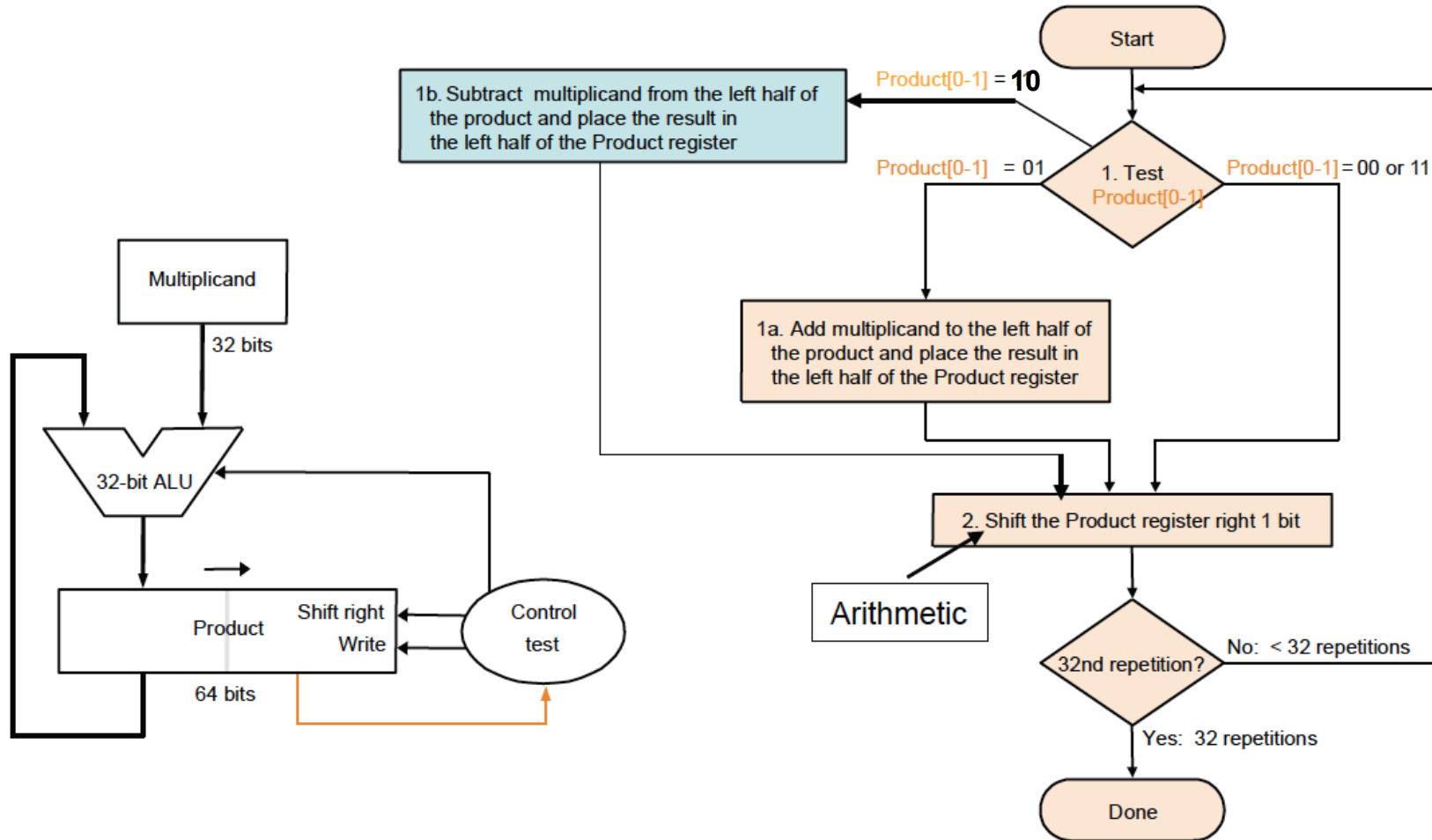
Signed Multiplication

- So far, we have multiplied unsigned numbers
- What about signed multiplication?
 - one solution: make both positive
 - leave out the sign bit, run for 31 steps
 - set sign bit negative if signs of inputs differ

Booth's Algorithm

- multiply two's complement signed numbers
- uses same hardware as before
- can also be used to reduce the number of steps

Booth's Algorithm



Booth's Example (-6 x -5)

- Multiplicand = -6 = 1010; -Multiplicand = 6 = 0110
- Multiplier = -5 = 1011

Iter.	Step	Product	Last	Action
0	0	0000 <u>101</u> (1 0)	0	Initialize
1	1.10	0110 <u>101</u> (1 0)	0	Start string: Subtract => Add 0110
1	2	0011 0 <u>10</u> (1 1)	1	Shift arithmetic
2	1.11	0011 0 <u>10</u> (1 1)	1	Middle string: nothing
2	2	0001 <u>101</u> (0 1)	1	Shift arithmetic
3	1.01	1 011 <u>101</u> (0 1)	1	End string: add 1010
3	2	1 101 110(<u>1</u> 0)	0	Shift <i>arithmetic</i>
4	1.10	0011 110(<u>1</u> 0)	0	Start string: Subtract => add 0110
4	2	0001 1110	0	Shift arithmetic

- Notes:
1. Multiplier in Product Register is underlined.
 2. Current/previous bits are in parentheses.
 3. Previous bit is initialized to 0

Example

- Determine $A \times B$ using the **third multiplication algorithm** for the following 8-bit numbers. Use $A = (0110\ 1111)_2$ for the multiplier and $B = (0001\ 1101)_2$ for the multiplicand.

Iter.	Step	Product	Multiplicand	Action
0	0	0000 0000 0110 1111	0001 1101	Initialize
1	1a.	0001 1101 0110 1111	0001 1101	Add
1	2	0000 1110 1011 0111	0001 1101	Shift
2	1a.	0010 1011 1011 0111	0001 1101	Add
2	2	0001 0101 1101 1011	0001 1101	Shift
3	1a.	0011 0010 1101 1011	0001 1101	Add
3	2	0001 1001 0110 1101	0001 1101	Shift
4	1a.	0011 0110 0110 1101	0001 1101	Add
4	2	0001 1011 0011 0110	0001 1101	Shift
5	1	0001 1011 0011 0110	0001 1101	No action
5	2	0000 1101 1001 1011	0001 1101	Shift
6	1a.	0010 1010 1001 1011	0001 1101	Add
6	2	0001 0101 0100 1101	0001 1101	Shift
7	1a.	0011 0010 0100 1101	0001 1101	Add
7	2	0001 1001 0010 0110	0001 1101	Shift
8	1	0001 1001 0010 0110	0001 1101	No action
8	2	0000 1100 1001 0011	0001 1101	Shift

Example

- Determine $A \times B$ using the **Booth's algorithm** for the following 8-bit numbers. Use $A = (0110\ 1111)_2$ for the multiplier and $B = (0001\ 1101)_2$ for the multiplicand.

- - **Multiplicand = 1110 0011**

Iter.	Step	Product	Previous	Action
0	0	0000 0000 0110 1111	0	Initialize
1.	1.10	1110 0011 0110 1111	0	Subtract
1	2	1111 0001 1011 0111	1	Shift
2	1.11	1111 0001 1011 0111	1	No action
2	2	1111 1000 1101 1011	1	Shift
3	1.11	1111 1000 1101 1011	1	No action
3	2	1111 1100 0110 1101	1	Shift
4	1.11	1111 1100 0110 1101	1	No action
4	2	1111 1110 0011 0110	1	Shift
5	1.01	0001 1011 0011 0110	1	Add
5	2	0000 1101 1001 1011	0	Shift
6	1.10	1111 0000 1001 1011	0	Subtract
6	2	1111 1000 0100 1101	1	Shift
7	1.11	1111 1000 0100 1101	1	No action
7	2	1111 1100 0010 0110	1	Shift
8	1.01	0001 1001 0010 0110	1	Add
8	2	0000 1100 1001 0011	0	Shift

Booth's Algorithm

- Originally for speed: Shifts are faster than add
- Key advantage today: Works properly for 2's complement numbers without requiring special fix for sign!



MESUT OZDAG, PH.D.
MESUT.OZDAG@UCF.EDU